

Apolium Platform

Technical Architecture

Data model, incentive engine, and how this stays correct at 5 lakh members.

Companion to the client-facing System Overview. This document assumes engineering knowledge. It covers the domain model, the PostgreSQL schema, the calculation engines, the batch/queue architecture, and the non-functional requirements that make the incentive plan auditable.

DOCUMENT	Technical Architecture & Data Model
AUDIENCE	Engineering team (internal)
COMPANION DOC	Apolium_System_Overview_CLIENT.docx
VERSION	1.0 · Draft
STATUS	Design — pre-implementation
OWNER	

CONTENTS

Table of Contents

0 1	System Context aur Constraints	5
1.1	Kya bana rahe hain	5
1.2	Scale target aur nikla hua load.....	5
1.3	Stack ka faisla	6
0 2	Domain Model	7
2.1	Do tree — inhe kabhi mix mat karo	7
2.2	Placement — extreme-chain rule.....	7
2.3	Point ke type	8
0 3	Data Model (PostgreSQL)	9
3.1	Conventions	9
3.2	Members aur dono tree	9
3.3	Closure tables.....	10
3.4	Products, bills aur point capture	11
3.5	Franchise, wallet aur limits.....	11
3.6	Price ladder — data, hardcode nahi.....	12
3.7	Point ledger — append-only, partitioned	13
3.8	Running balance (denormalised, transactional)	14
3.9	Money aur points ki precision	15
0 4	Point Fan-out Engine.....	17
4.1	Flow	17
4.2	Worker ka logic.....	17
4.3	Hot-row problem — aur uska fix	18
4.4	Correctness ke points	19
0 5	BPI Matching Engine (Nightly Close).....	21
5.1	Algorithm	21
5.2	Capping table (seed data).....	22

5.3	5 lakh members pe batch execution	23
5.4	Weekly payout	24
0 6	Mentor's, Startup aur Achievers Engine	25
6.1	Mentor's Incentive (sponsor tree, 5 levels × 7%)	25
6.2	Startup Incentive (one-time milestones)	25
6.3	Achievers (monthly)	26
0 7	Non-Functional Requirements	27
7.1	Correctness aur auditability	27
7.2	Performance target	27
7.3	Security	27
7.4	Operations	28
7.5	Read replica	29
7.6	Storage ka anumaan	29
0 8	Incentive Engine ki Testing	30
8.1	Zaroori test fixtures	30
8.2	Load aur soak testing	30
0 9	Open Technical Items	32
9.1	Jo decisions lock ho chuke	32
1 0	Repo Structure, Apps & Deployment	34
10.1	Monorepo layout	34
10.2	Apps — kaun kya karta hai	34
10.3	Domain aur SEO isolation	35
10.4	Auth — teen alag audience, ek hi backend	36
10.5	Future-proofing — naya kaam kaise add hoga	36
10.6	Deployment	37
1 1	Research Basis aur References	38
11.1	Tree-storage evaluation	38
11.2	Recursive CTE hot path pe kyun nahi	38
11.3	Sources	39

11.4 Ek line me	40
-----------------------	----

01 System Context aur Constraints

1.1 Kya bana rahe hain

Do jude hue domain, ek hi transaction stream pe:

DOMAIN	SHAPE	CORE OPERATIONS	VOLUME DRIVER
Distribution (ERP-like)	Level hierarchy WH → MWH → L1 → L2 → L3	Fund request, stock request, billing, GST invoicing, wallet/stock limits	Franchise count (hundreds)
Network (MLM)	Binary placement tree + separate sponsor DAG	Placement, point fan-out, daily matching, capping, payouts	Member count (lakhs)

ARCHITECTURE KA SABSE BADA POINT

Ek **bill** row hi **ekmatra** write hai jise dono domain dekhte hain. Distribution use **synchronously** consume karta hai (stock, GST, wallet). Network use **asynchronously** (point fan-out).

Yahi async boundary system ko 5 lakh members pe zinda rakhti hai — cashier ko kabhi 40-row tree walk ka wait nahi karna chahiye.

1.2 Scale target aur nikla hua load

5,00,000 MEMBERS (DESIGN TARGET)	~40 MAX TREE DEPTH	2–5 M LEDGER ROWS / MONTH	< 2 h NIGHTLY CLOSE WINDOW
--	------------------------------	-------------------------------------	---

Load derivation. Assume 5,00,000 members, 15% transacting per month (75,000 bills/month $\approx$ 2,500/day, peaking $\sim 10\times$ at month-end). Each bill fans out to the buyer + sponsor + every ancestor. Balanced-tree depth for 5 lakh nodes is $\log_2(500000) \approx 19$; real MLM trees are heavily skewed by the extreme-chain placement rule, so plan for **depth 40–60** on the long legs.

QUANTITY	CALCULATION	RESULT
Point-ledger rows / bill	1 self + 1 direct + depth (~40) team rows	~42 rows
Point-ledger rows / month	75,000 bills $\times$ 42	~3.2 million
Rows / year	3.2M $\times$ 12	~38 million
Daily close: accounts touched	members with non-zero volume	up to 5,00,000

Peak fan-out burst

month-end 25,000 bills/day × 42

~1.05 M writes/day**ISLIYE NAIVE DESIGN KAAM NAHI KAREGA**

A recursive **WITH RECURSIVE** walk per bill at depth 40, 25,000 times on the last day of the month, inside the billing transaction, will not hold. Neither will recomputing leg volumes by summing subtrees.

The design below therefore uses: **(a)** a closure table for O(1) ancestor lookup, **(b)** denormalised per-leg running counters, **(c)** queue-driven fan-out with idempotency keys, **(d)** an append-only ledger that is never mutated.

1.3 Stack ka faisla

LAYER	CHOICE	RATIONALE
Database	PostgreSQL 16+	ACID, NUMERIC money, recursive CTEs, RANGE partitioning, mature ops. Boring on purpose.
Backend	Node 20 + TypeScript (NestJS) or Python 3.12 + FastAPI	Either is fine; TS keeps one language with the web app. Pick one and do not mix.
ORM	Prisma 7 (TS) / SQLAlchemy 2 (Py)	Migrations + typed access. Raw SQL for the closure/fan-out hot paths.
Queue	BullMQ (Redis) or Celery	Point fan-out, nightly close, payout batches, notifications.
Cache	Redis	Session, rate limit, hot dashboard aggregates, distributed locks.
Pooling	PgBouncer (transaction mode)	Mandatory. Worker fleet will otherwise exhaust max_connections .
Reporting	Read replica	Dashboards, genealogy browsing and exports never touch the primary.

02 Domain Model

2.1 Do tree — inhe kabhi mix mat karo

	PLACEMENT TREE (BINARY)	SPONSOR TREE (N-ARY)
Column	<code>placement_parent_id + leg</code>	<code>sponsor_id</code>
Arity	Exactly 2 children max (LEFT / RIGHT)	Unbounded children
Drives	Team points, BPI matching, leg volumes	Direct points, Startup Incentive, Mentor's Incentive
Traversal need	All ancestors (fan-out) + subtree volume	Up to 5 levels only
Storage	Closure table (<code>placement_closure</code>)	Closure table capped at depth 5 (<code>sponsor_closure</code>)

SPONSOR CLOSURE BOUNDED HAI — ISKA FAYDA UTHAO

Mentor's Incentive only reaches **5 levels**. Storing `sponsor_closure` rows only for `depth <= 5` caps that table at roughly `members × 5` rows (~2.5 M) instead of the full transitive closure. Placement closure has no such cap and is the larger structure.

2.2 Placement — extreme-chain rule

Placement is **not** 'nearest open slot'. Given a sponsor node and a chosen leg, the algorithm walks that single direction until it finds a node with no child on that side.

PSEUDOCODE

```
function findPlacementSlot(startId, leg) {
  // leg ∈ {'LEFT', 'RIGHT'} — never switches direction
  let cur = startId;
  while (true) {
    const child = getChild(cur, leg); // placement_parent_id = cur AND leg = leg
    if (!child) return cur;           // attach new member here, on `leg`
    cur = child;
  }
}
```

IMPLEMENTATION NOTES

This is $O(\text{depth})$ with an index on (`placement_parent_id`, `leg`). Cheap — it runs once per registration.

Concurrency: two simultaneous registrations under the same sponsor+leg will race to the same slot.

Take a `pg_advisory_xact_lock(hashtext('place:' || sponsor_id || ':' || leg))` for the duration of

the placement transaction, or enforce a `UNIQUE (placement_parent_id, leg)` constraint and retry on conflict. **Do both.**

Maintain an extremity pointer — cache the current tip of each (node, leg) chain in a `placement_tip` table so registration is $O(1)$ instead of $O(\text{depth})$. Update it in the same transaction.

2.3 Point ke type

POINT	RECIPIENT	CARDINALITY PER BILL	CONSUMED BY
<code>SELF</code>	Buyer	1	Capping level determination
<code>DIRECT</code>	Buyer's sponsor	0 or 1	Startup Incentive milestones
<code>TEAM_LEFT</code> / <code>TEAM_RIGHT</code>	Every placement ancestor	= tree depth (~40)	BPI daily matching

CONFIRMED RULES

Buyer receives **`SELF` only** — no team point for themselves.

Sponsor receives `DIRECT` **and**, if they are also a placement ancestor, `TEAM_*` as well. These are two independent credits from the same bill.

Side selection: for each ancestor A, the side is determined by **which of A's children the buyer's branch descends through** — resolved from the closure table, not by re-walking.

03 Data Model (PostgreSQL)

3.1 Conventions

- **UUID** primary keys (`gen_random_uuid()`, core since PG13) except high-volume append tables which use **BIGSERIAL**.
- Every table: `created_at`, `updated_at` (`TIMESTAMPTZ`), and `deleted_at` for soft delete where applicable.
- **Money is `BIGINT` in paise. Points are `BIGINT` in milli-PV (×1000). Rates are basis points.** Dekho §3.9 — ye pehle wale `NUMERIC(18,4)` draft ko replace karta hai.
- All timestamps `TIMESTAMPTZ`; business dates (`business_date`) stored as `DATE` in IST to make closing deterministic.
- Every foreign key column carries its own index — PostgreSQL does not create it for you.
- Ledger and transaction tables are **append-only** and **RANGE-partitioned by month**.

3.2 Members aur dono tree

```
SQL
CREATE TABLE members (
  id                UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  member_code       TEXT NOT NULL UNIQUE,      -- human-facing ID used on bills
  full_name         TEXT NOT NULL,
  mobile            TEXT NOT NULL UNIQUE,
  kyc_status        TEXT NOT NULL DEFAULT 'PENDING'
                  CHECK (kyc_status IN ('PENDING','VERIFIED','REJECTED')),
  activated_at      TIMESTAMPTZ,              -- set when 250 self-PV is crossed

  sponsor_id        UUID REFERENCES members(id),      -- WHO referred
  placement_parent_id UUID REFERENCES members(id),    -- WHERE in binary tree
  leg               TEXT CHECK (leg IN ('LEFT','RIGHT')), -- side under the parent
  depth            INT  NOT NULL DEFAULT 0,          -- distance from root

  created_at        TIMESTAMPTZ NOT NULL DEFAULT NOW(),
  updated_at        TIMESTAMPTZ NOT NULL DEFAULT NOW(),
  deleted_at        TIMESTAMPTZ,

  CONSTRAINT one_child_per_leg UNIQUE (placement_parent_id, leg)
);

CREATE INDEX members_sponsor_idx  ON members(sponsor_id);
CREATE INDEX members_placement_idx ON members(placement_parent_id, leg);
CREATE INDEX members_code_idx     ON members(member_code);
```

`UNIQUE (PLACEMENT\_PARENT\_ID, LEG)` HI ASLI SAFETY NET HAI

This single constraint makes it **physically impossible** to attach two members to the same leg of the same parent, regardless of application bugs or concurrent requests. The advisory lock is the performance

path; this constraint is the correctness guarantee.

3.3 Closure tables

```
SQL
-- Every (ancestor, descendant) pair, including self at depth 0.
CREATE TABLE placement_closure (
  ancestor_id  UUID NOT NULL REFERENCES members(id),
  descendant_id UUID NOT NULL REFERENCES members(id),
  depth        INT  NOT NULL,
  branch_leg   TEXT NOT NULL CHECK (branch_leg IN ('SELF','LEFT','RIGHT')),
  -- branch_leg = which of ancestor's legs this descendant hangs under.
  -- Precomputed so point fan-out never has to re-derive the side.
  PRIMARY KEY (ancestor_id, descendant_id)
);

CREATE INDEX pc_desc_idx ON placement_closure(descendant_id, depth);
CREATE INDEX pc_anc_leg_idx ON placement_closure(ancestor_id, branch_leg);

-- Sponsor closure, capped at 5 levels (Mentor's Incentive maximum).
CREATE TABLE sponsor_closure (
  ancestor_id  UUID NOT NULL REFERENCES members(id),
  descendant_id UUID NOT NULL REFERENCES members(id),
  level        INT  NOT NULL CHECK (level BETWEEN 1 AND 5),
  PRIMARY KEY (ancestor_id, descendant_id)
);
CREATE INDEX sc_desc_idx ON sponsor_closure(descendant_id, level);
```

Insert on registration — one statement each, no recursion at read time:

```
SQL
-- New member N attaches under parent P on leg L.
INSERT INTO placement_closure (ancestor_id, descendant_id, depth, branch_leg)
SELECT pc.ancestor_id, :new_id, pc.depth + 1,
       CASE WHEN pc.ancestor_id = :parent_id THEN :leg ELSE pc.branch_leg END
FROM   placement_closure pc
WHERE  pc.descendant_id = :parent_id
UNION ALL
SELECT :new_id, :new_id, 0, 'SELF';
```

COST KA HISAAB

Rows inserted per registration = **depth + 1**. At depth 40 that is 41 rows — trivial for a once-per-member operation.

Total table size at 5 lakh members with average depth ~25: **≈ 12.5 million rows**. With UUID keys that is roughly **1–1.5 GB including indexes** — comfortable for a single PostgreSQL node.

The payoff: 'give me every ancestor of member X and which side X is on' becomes a **single index scan** on `pc_desc_idx`, instead of a 40-iteration recursive CTE, on every one of 3.2 M monthly fan-outs.

3.4 Products, bills aur point capture

```
SQL
CREATE TABLE products (
  id          UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  product_code TEXT NOT NULL UNIQUE,          -- Ag-Rakshak123
  name        TEXT NOT NULL,
  category_id UUID NOT NULL REFERENCES categories(id),
  mrp         NUMERIC(18,4) NOT NULL,
  msp         NUMERIC(18,4) NOT NULL,          -- member selling price
  gst_percent NUMERIC(5,2) NOT NULL,
  pv          NUMERIC(18,4) NOT NULL DEFAULT 0,
  gv          NUMERIC(18,4) NOT NULL DEFAULT 0,
  cv          NUMERIC(18,4) NOT NULL DEFAULT 0,
  is_active   BOOLEAN NOT NULL DEFAULT TRUE
);

-- Level-wise price ladder (WH 670 → MWH 700 → L1 725 → L2 750 → member 800)
CREATE TABLE product_level_prices (
  product_id UUID NOT NULL REFERENCES products(id),
  level_code  TEXT NOT NULL,                  -- WH | MWH | L1 | L2 | L3 | MEMBER
  price      NUMERIC(18,4) NOT NULL,
  PRIMARY KEY (product_id, level_code)
);

CREATE TABLE bills (
  id          UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  bill_no     TEXT NOT NULL UNIQUE,
  franchise_id UUID NOT NULL REFERENCES franchises(id),
  member_id   UUID NOT NULL REFERENCES members(id),  -- NOT NULL: no ID, no bill
  business_date DATE NOT NULL,
  subtotal    NUMERIC(18,4) NOT NULL,
  gst_amount  NUMERIC(18,4) NOT NULL,
  total       NUMERIC(18,4) NOT NULL,
  total_pv    NUMERIC(18,4) NOT NULL,
  total_gv    NUMERIC(18,4) NOT NULL,
  status      TEXT NOT NULL DEFAULT 'CONFIRMED'
              CHECK (status IN ('DRAFT', 'CONFIRMED', 'CANCELLED')),
  fanout_status TEXT NOT NULL DEFAULT 'PENDING'
              CHECK (fanout_status IN ('PENDING', 'DONE', 'FAILED')),
  created_at  TIMESTAMPTZ NOT NULL DEFAULT NOW()
);
CREATE INDEX bills_fanout_idx ON bills(fanout_status) WHERE fanout_status = 'PENDING';
```

‘MEMBER_ID NOT NULL’ — BUSINESS RULE DATABASE ME HI ENFORCE

The 'no ID, no bill' rule is not left to the UI. A bill row physically cannot exist without a member. This is the difference between a rule and a **guarantee**.

3.5 Franchise, wallet aur limits

```
SQL
CREATE TABLE franchises (
  id          UUID PRIMARY KEY DEFAULT gen_random_uuid(),
```

```

code          TEXT NOT NULL UNIQUE,
level_code    TEXT NOT NULL,      -- WH | MWH | L1 | L1_SPECIAL | L2 | L3
parent_id     UUID REFERENCES franchises(id),
gstin         TEXT,
-- limits: kis type ka franchise hai uspe depend karta hai
minus_limit   BIGINT NOT NULL DEFAULT 0,  -- MWH / L1 / L1_SPECIAL (paise)
stock_limit   BIGINT NOT NULL DEFAULT 0,  -- L2 aur neeche (paise)
can_approve_funds BOOLEAN NOT NULL DEFAULT FALSE, -- L1, L1_SPECIAL, MWH
is_active     BOOLEAN NOT NULL DEFAULT TRUE
);

CREATE TABLE franchise_balance (
  franchise_id  UUID PRIMARY KEY REFERENCES franchises(id),
  main_wallet   BIGINT NOT NULL DEFAULT 0,  -- minus ja sakta (MWH/L1)
  commission_wallet BIGINT NOT NULL DEFAULT 0, -- transferable -> main
  retail_profit BIGINT NOT NULL DEFAULT 0,  -- VIEW ONLY, kabhi debit nahi
  stock_value   BIGINT NOT NULL DEFAULT 0
) WITH (fillfactor = 85);

```

LEVEL	LIMIT KA TYPE	ENFORCE KAHAN
MWH, L1, L1_SPECIAL	minus_limit — main wallet itna negative ja sakta	Stock/fund request pe check
L2 aur neeche	stock_limit — stock is se neeche gaya to billing block	Bill create hone se pehle guard

L1-SPECIAL — ALAG FLAG, ALAG BEHAVIOUR

L1_SPECIAL ka **koi limit nahi** — L2 jaisa chalta hai (pehle fund, phir stock). Par `can_approve_funds = TRUE` — ye doosron ki fund request approve kar sakta hai, L1 aur MWH ki tarah.

Isliye level ek ``level_code`` + **capability flags** ka combination hai, hardcoded enum branching nahi. Kal naya level (L4) aaya to sirf ek row add hogi.

RETAIL PROFIT WALLET — DB ME HI LOCK KARO

`retail_profit` sirf **credit** hota hai, kabhi debit nahi. Ye rule application code pe mat chhodo — [CHECK](#) constraint ya trigger se enforce karo, warna ek din koi transfer endpoint isse nikal lega.

3.6 Price ladder — data, hardcode nahi

HOP	PRICE	MARGIN
WH → L1	₹700	—
L1 → L2	₹725	L1 = ₹25
L2 → L3	₹750	L2 = ₹25

L3 → member	₹800 (MSP)	L3 = ₹50
(agar MWH ho) WH → MWH	₹685	MWH = ₹15 (700 – 685)

⚠ SOURCE CONFLICT — CLIENT SE CONFIRM KARNA HAI

Module spec (01-franchise-module/MODULES.md §2.3, §3.2) me: **L1 700 · L2 725 · L3 750 · member 800**, aur MWH 685 pe kharidta hai (margin 15).

Client ke handwritten note me: **670 · 700 · 750 · 800** — ek level ka shift hai.

Humne **module spec** ko liya hai kyunki wo likha hua spec hai. **Ye client-facing open question hai** — ladder pe poora commission depend karta hai. Prices `product_level_prices` table me hain, isliye correction sirf data change hai, code change nahi.

MISSING-LEVEL ROLL-UP

Agar chain me koi level nahi hai, uska margin **upar wale existing level** me jud jaata hai. Total distributed margin hamesha ₹100 rehta hai.

Implementation: bill ke time chain ko upar walk karo, jo levels **exist** karte hain unhe collect karo, aur missing levels ka margin next-existing-ancestor ko de do. Ye pure function hai → `packages/core` me, taaki test ho sake.

3.7 Point ledger — append-only, partitioned

```
SQL
CREATE TABLE point_ledger (
  id                BIGSERIAL,
  member_id         UUID NOT NULL,
  bill_id           UUID NOT NULL,
  point_type        TEXT NOT NULL
    CHECK (point_type IN ('SELF', 'DIRECT', 'TEAM_LEFT', 'TEAM_RIGHT')),
  pv                NUMERIC(18,4) NOT NULL,
  gv                NUMERIC(18,4) NOT NULL DEFAULT 0,
  source_member     UUID NOT NULL,      -- the buyer who generated it
  business_date     DATE NOT NULL,
  created_at        TIMESTAMPTZ NOT NULL DEFAULT NOW(),
  PRIMARY KEY (id, business_date),
  UNIQUE (bill_id, member_id, point_type, business_date) -- idempotency key
) PARTITION BY RANGE (business_date);

CREATE TABLE point_ledger_2026_08 PARTITION OF point_ledger
  FOR VALUES FROM ('2026-08-01') TO ('2026-09-01');

CREATE INDEX pl_member_date_idx
  ON point_ledger(member_id, business_date, point_type);
```

TEEN SOCH-SAMAJH KE LIYE GAYE FAISLE

``UNIQUE (bill_id, member_id, point_type, business_date)`` — this is the idempotency key. A retried fan-out job hits a conflict instead of double-crediting. Use `ON CONFLICT DO NOTHING`.

Partition by ``business_date`` — the nightly close reads exactly one partition; old months can be detached and archived without touching live data. Automate partition creation with `pg_partman`; an insert with no matching partition is a hard error.

Never ``UPDATE`` this table. A correction is a new compensating row, not an edit. This is what makes the plan defensible if a member disputes a payout.

3.8 Running balance (denormalised, transactional)

```
SQL
CREATE TABLE member_volume (
  member_id      UUID PRIMARY KEY REFERENCES members(id),
  self_pv_total  NUMERIC(18,4) NOT NULL DEFAULT 0,    -- drives capping level
  self_pv_month  NUMERIC(18,4) NOT NULL DEFAULT 0,    -- monthly maintenance check
  direct_pv_total NUMERIC(18,4) NOT NULL DEFAULT 0,    -- Startup Incentive milestones
  carry_left     NUMERIC(18,4) NOT NULL DEFAULT 0,    -- BPI carry-forward
  carry_right    NUMERIC(18,4) NOT NULL DEFAULT 0,
  today_left     NUMERIC(18,4) NOT NULL DEFAULT 0,    -- reset by nightly close
  today_right    NUMERIC(18,4) NOT NULL DEFAULT 0,
  capping_level  INT NOT NULL DEFAULT 0,
  updated_at     TIMESTAMPTZ NOT NULL DEFAULT NOW()
);
```

DENORMALISE KYUN KAR RAHE HAIN

Recomputing a leg's volume means summing an entire subtree. At 5 lakh members the root's weaker leg is ~250,000 members — a 250k-row aggregate on every page load and every commission calculation. Not viable at any depth for the top ~100 members.

Instead these counters are incremented **in the same transaction** as the ledger insert. The ledger remains the source of truth; `member_volume` is a materialised projection.

Industry check: Exigo, an enterprise direct-selling platform, advertises volume updates "in near real-time in just **1 to 5 minutes**" — even a market leader treats volume propagation as an asynchronous batched pipeline, not a synchronous write.

COUNTER TABLE KE DO PHYSICAL RULE

``WITH (fillfactor = 85)``. Leaving free space in each page lets PostgreSQL do **HOT (heap-only tuple) updates** — the row is rewritten in place without touching indexes. Adyen published production numbers for exactly this change: HOT-update rate **63.9% → 92.2%**, peak dead rows ~70M → ~40M, table **30 GB → 24 GB**, cluster WAL **3.15 → 2.80 TB/day**.

Never index a counter column. HOT does not apply when an updated column is covered by an index.

One index on `today_left` turns every counter bump from one page write into two or more. `member_volume` carries a primary key and nothing else.

3.9 Money aur points ki precision

An earlier draft of this document specified `NUMERIC(18,4)`. That is correct but slow, and it leaves a rounding-mode gap between services. The final decision is **integers in minor units**, which is standard practice at Stripe and Modern Treasury.

QUANTITY	TYPE	RATIONALE
Money (wallet, commission, payout)	<code>BIGINT</code> paise	Exact, 8 bytes, fast to <code>SUM()</code> over millions of ledger rows. Range covers ~92 quadrillion — not a constraint.
Points (PV / GV / CV)	<code>BIGINT</code> milli-PV (×1000)	Plan documents carry 2–3 decimals on points. Scaling to integers keeps matching, capping and carry-forward arithmetic exact.
Rates (80%, 7%)	<code>INTEGER</code> basis points (8000, 700)	Stored in a date-effective config table. Keeps the whole split in integer arithmetic.
Display / API	Format at the edge	One formatting layer from integer + exponent. No drift between services.

FLOAT KYUN NAHI — EK CONCRETE EXAMPLE

A 32-bit float must approximate ₹25,474,937.47 as ₹25,474,936.32 — **off by ₹1.15**. Worse, default rounding modes differ by language (Ruby half-away-from-zero, Python 3 half-even, JavaScript half-toward-+∞), so two services computing the same split legitimately disagree.

PostgreSQL's own documentation is explicit: "If you require exact storage and calculations (such as for monetary amounts), use the `numeric` type instead" — and equally that "calculations with `numeric` values are very slow compared to the integer types." Integers in minor units satisfy both.

Splitting a pool without creating or destroying money

80% + 7% + 13% of a pool, each rounded independently, does **not** sum to the pool. Use the **largest-remainder method** — the standard in payroll and dividend distribution.

ALLOCATION

Matched value = ₹1,234.56 = 123,456 paise. Splits: matching 80%, mentor 7%, company 13%.

```

floor(123456 × 8000 / 10000) = 98,764    remainder .80
floor(123456 × 700 / 10000) = 8,641      remainder .92
floor(123456 × 1300 / 10000) = 16,049    remainder .28
subtotal = 123,454                      → 2 paise unallocated

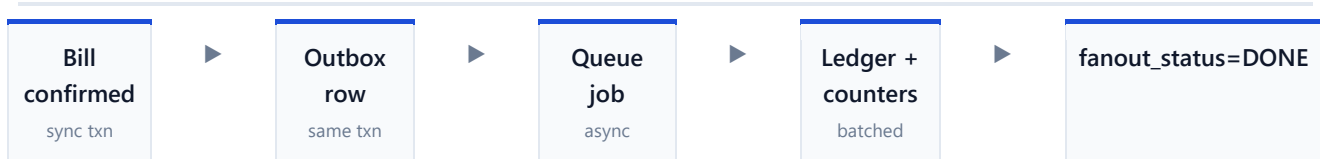
```

```
allocate to largest remainders: mentor (.92) +1 → 8,642
                                matching(.80) +1 → 98,765
                                total    = 123,456 ✓ exactly conserved
```

- **Compute every split from the same integer pool, in one function, in one transaction.** Never `round(x*0.80)` in one service and `round(x*0.07)` in another.
- **Assert `SUM(splits) == pool`** before writing the ledger; fail the transaction otherwise. Cheap invariant, catches plan-config mistakes.
- **Fix a deterministic tie-break** (e.g. `reason_code` alphabetically) so equal remainders allocate identically on every re-run — otherwise an 'idempotent' close produces different output on retry.
- **Capping is applied before splitting**, and the capped-off amount is written as an explicit `CAPPING_WASHOUT` entry — never silently dropped, or the ledger stops balancing.

04 Point Fan-out Engine

4.1 Flow



TRANSACTIONAL OUTBOX — SEEDHA ENQUEUE KYUN NAHI?

If you `INSERT bill` then `queue.add(job)`, a crash between the two silently loses the points and no one notices until a member complains.

Instead, write an `outbox` row **inside the billing transaction**. A relay worker polls the outbox and enqueues. Now the bill and its intent to fan out commit atomically — the job can be delayed, never lost.

`bills.fanout_status` gives a cheap safety net: a reconciliation job scans for `PENDING` bills older than N minutes and re-enqueues them.

4.2 Worker ka logic

TYPESCRIPT

```

async function fanOutBill(billId) {
  const bill = await getBill(billId);
  if (bill.status !== 'CONFIRMED') return;

  // 1. All ancestors + which side, in ONE index scan (no recursion).
  const ancestors = await sql`
    SELECT ancestor_id, branch_leg, depth
    FROM   placement_closure
    WHERE  descendant_id = ${bill.member_id} AND depth > 0`;

  const rows = [
    { member: bill.member_id, type: 'SELF', pv: bill.total_pv },
    ...(bill.sponsor_id
      ? [{ member: bill.sponsor_id, type: 'DIRECT', pv: bill.total_pv }] : []),
    ...ancestors.map(a => ({
      member: a.ancestor_id,
      type:   a.branch_leg === 'LEFT' ? 'TEAM_LEFT' : 'TEAM_RIGHT',
      pv:     bill.total_pv,
    })),
  ];

  await db.transaction(async tx => {
    // 2. One multi-row INSERT. Idempotent via the unique key.
    await tx.insertMany('point_ledger', rows, { onConflict: 'DO NOTHING' });

    // 3. Batch-update the projections in a single statement.
  });
}
  
```

```

await tx.sql`
  UPDATE member_volume mv SET
    today_left = mv.today_left + v.left_pv,
    today_right = mv.today_right + v.right_pv,
    self_pv_total = mv.self_pv_total + v.self_pv,
    direct_pv_total = mv.direct_pv_total + v.direct_pv
  FROM (VALUES ${values}) AS v(member_id, left_pv, right_pv, self_pv, direct_pv)
  WHERE mv.member_id = v.member_id::uuid`;

await tx.sql`UPDATE bills SET fanout_status='DONE' WHERE id=${billId}`;
});
}

```

4.3 Hot-row problem — aur uska fix

THROUGHPUT NAHI — YE CHEEZ NAIVE DESIGN KO MARTI HAI

The root member and the top ~50 members sit on the ancestor path of **literally every purchase in the company**. This is the social-graph 'celebrity problem', except here it is structural and permanent.

At 20 purchases/second, one-bill-at-a-time fan-out issues **20 serialised `UPDATE`s per second against the root row**. PostgreSQL queues every transaction waiting on that row lock, so each purchase blocks on every other purchase. Raw throughput is fine — **lock contention is the failure mode**.

The fix is to **expand a whole batch first, aggregate by ancestor, then write once**. Because the upper levels of the tree are shared by everyone, grouping collapses the work dramatically.

```

SQL
WITH batch AS (
  SELECT o.id AS outbox_id, e.bill_id, e.member_id, e.pv, e.business_date
  FROM   outbox o
  JOIN   point_event e ON e.id = (o.payload->>'point_event_id')::bigint
  WHERE  o.published_at IS NULL
  ORDER BY o.id
  LIMIT  1000
  FOR UPDATE SKIP LOCKED           -- multi-worker safe, no contention
),
expanded AS (                       -- closure table does the fan-out, set-based
  SELECT pc.ancestor_id, b.business_date,
         SUM(b.pv) FILTER (WHERE pc.branch_leg = 'LEFT') AS left_pv,
         SUM(b.pv) FILTER (WHERE pc.branch_leg = 'RIGHT') AS right_pv
  FROM   batch b
  JOIN   placement_closure pc
        ON pc.descendant_id = b.member_id AND pc.depth > 0
  GROUP BY pc.ancestor_id, b.business_date
)
UPDATE member_volume v
SET   today_left = v.today_left + e.left_pv,
      today_right = v.today_right + e.right_pv
FROM   expanded e
WHERE  v.member_id = e.ancestor_id;

```

METRIC

PER-BILL FAN-OUT

BATCH-AGGREGATED FAN-

		OUT
Rows written per 1,000 bills	~40,000	~8,000–15,000
Write amplification	~40×	~10×
Updates to the root row	1,000	1
Lock contention at the top	Severe, permanent	Eliminated by construction

DO IMPLEMENTATION DETAIL JO MATTER KARTI HAIN

Drive the batch with `UPDATE ... FROM unnest($1::bigint[], $2::bigint[])`` rather than a `VALUES` list. It uses a fixed 2 parameters regardless of batch size — dodging PostgreSQL's **32,767 parameter limit** — and benchmarks show `UNNEST` at **2.1×** faster than `VALUES` at batch size 1,000 (5.0× on wider rows), the saving coming from planning time.

Honest caveat: a queue does not raise the database's sustained write throughput — the worker still writes to the same PostgreSQL instance. The win is **latency isolation, batch aggregation and spike absorption**, not extra capacity.

4.4 Correctness ke points

CONCERN	HANDLING
Idempotency	<code>UNIQUE (bill_id, member_id, point_type, business_date) + ON CONFLICT DO NOTHING.</code> Safe to retry any number of times.
Outbox delivery	The outbox guarantees at-least-once , not exactly-once — a relay that publishes then crashes will republish. Consumers must be idempotent. That is why the unique key above is not optional.
Outbox hygiene	Partial index <code>(id) WHERE published_at IS NULL</code> keeps the poller O(backlog) not O(history). Partition the outbox by day and drop old partitions.
Ordering	Not required — point credits are commutative. No per-member serialisation needed.
Deadlocks	Sort the aggregated ancestor list by <code>member_id</code> before issuing the <code>UPDATE</code> so all workers take row locks in the same order.
Cancellation	A cancelled bill inserts compensating negative rows — original rows are never deleted or edited.

OPEN — CLIENT SE DECISION CHAHIYE

Bill cancellation / stock return: the client has not yet defined whether already-paid incentives are clawed back. The schema supports compensating entries either way, but the **policy** must be decided before Phase 5 goes live. See §9 of the client document.

05 BPI Matching Engine (Nightly Close)

5.1 Algorithm

Runs once per business date, per member with non-zero volume. This is the most business-critical code in the system — it is specified here precisely because every rule was hard-won.

```

TYPESCRIPT
function closeDay(m) {
    // m = member_volume row
    const cap = CAP_TABLE[m.capping_level].dailyCap; // 0 if level 0

    const availL = m.carry_left + m.today_left;
    const availR = m.carry_right + m.today_right;

    // Match = min of the two sides, floored to a 1000 multiple, then capped.
    const rawMatch = Math.floor(Math.min(availL, availR) / 1000) * 1000;
    const matched = Math.min(rawMatch, cap);
    const payout = matched * 0.80;

    const remL = availL - matched;
    const remR = availR - matched;

    // WASHOUT — decided by comparing OLD CARRY, not the totals.
    // bigger old carry -> nothing washes (its today business carries too)
    // smaller old carry -> everything remaining on that side washes,
    // but ONLY if that side received business today
    // no payout -> nothing washes at all
    let washL = 0, washR = 0;
    if (matched > 0) {
        if (m.carry_left < m.carry_right && m.today_left > 0) washL = remL;
        if (m.carry_right < m.carry_left && m.today_right > 0) washR = remR;
    }

    return {
        matched, payout,
        washL, washR,
        carryLeft: remL - washL,
        carryRight: remR - washR,
    };
}

```

CLIENT KE APNE EXAMPLE SE VERIFY KIYA

Day 1 — carry L 1,59,400 / R 2,047; no new business; cap 2,000 → matched 2,000, payout 1,600, **carry L 1,57,400 · carry R 47**.

Day 2 — today R 2,394 → matched 2,000, payout 1,600, Right side washes, **carry L 1,55,400 · carry R 0**.

Both days reproduce the client's handwritten figures exactly. The `today_* > 0` guard is the **assumption** that makes Day 1's 47 survive — flagged for client confirmation.

5.2 Capping table (seed data)

LEVEL	SELF PV ≥	DAILY CAP (PV)	MAX PAYOUT	MONTHLY MAINTENANCE
0	0	0	no payout	—
1	250	1,000	800	0
2	500	2,000	1,600	50
3	750	3,000	2,400	100
4	1,000	4,000	3,200	150
5	2,000	5,000	4,000	200
6	3,000	7,000	5,600	300
7	4,000	10,000	8,000	400
8	5,000	12,000	9,600	500

Activation aur monthly maintenance

RULE	IMPLEMENTATION
ID activate karne ke liye 250 PV self-purchase	Fan-out ke baad <code>self_pv_total >= 250</code> check → <code>members.activated_at</code> set. Jab tak <code>activated_at IS NULL</code> , close job us member ko skip karta hai.
Registration free · KYC zaroori · min age 18	<code>kyc_status</code> VERIFIED hona chahiye payout ke liye. Age validation registration pe.
Level bachaye rakhne ko har mahine minimum PV	Month-end job: <code>self_pv_month</code> ko slab ke <code>monthly_maintain</code> se compare karo → level barkarar ya girao. <code>self_pv_month</code> phir 0.
Level 0 (<250 PV) → koi payout nahi	Close job me pehla guard. Team volume kitna bhi ho, <code>matched = 0</code> .

ISKO DATA ME RAKHO, CODE ME NAHI

Put the capping table in a `capping_levels` table with an `effective_from` date. The plan **will** change. A code deploy must never be required to adjust a slab — and historical closes must keep using the slab that was in force on that business date.

5.3 5 lakh members pe batch execution

SCHEMA

```
closing_run      (id, business_date, status, config_hash, started_at, finished_at)
closing_run_chunk (run_id, chunk_no, member_lo, member_hi, status, entries_written)
```

PROPERTY	DESIGN
Selection	Only members with <code>today_left > 0 OR today_right > 0 OR carry_left > 0 OR carry_right > 0</code> . Partial index on that predicate.
Chunking	5,000 members per chunk → 100 chunks. Each chunk is one transaction : read volumes → match → apply capping → write ledger → write carry → mark chunk DONE .
Restartability	Each committed chunk moves the high-water mark forward. A crashed run resumes at the first incomplete chunk; completed chunks are never redone.
Idempotency	<code>UNIQUE (closing_run_id, member_id, reason_code)</code> on ledger entries. Re-running chunk 47 after a crash hits the constraint and writes nothing.
Parallelism	8 workers pulling chunks with <code>FOR UPDATE SKIP LOCKED</code> . Members are independent — no cross-chunk coordination needed.
Locking	<code>pg_advisory_xact_lock(hashtext('close:' business_date))</code> — transaction-scoped only . Session-level advisory locks are broken by PgBouncer transaction pooling (§7.4).
Frozen inputs	<code>config_hash</code> records the plan version, capping table and rates in force. A re-run with a different plan version must be a new run , never a resume.
Reversal	A bad run is corrected by a new run of contra entries referencing the original <code>closing_run_id</code> . Never by mutating the original.
Estimated runtime	~2 ms of work per member → ~17 min single-threaded, ~2–3 min at 8-way parallelism . Industry benchmark for a full commission run is under 10 minutes ; budget a 2-hour window.

YE DESIGN JIS FAILURE KO ROKTA HAI

From the batch-processing literature, the risk is stated bluntly: "When a simple `INSERT INTO SELECT` statement lacks a deduplication guard, retrying a failed job can lead to **catastrophic duplicate payouts**."

Every one of the properties above exists to make a retry provably free of side effects. This is the single most important piece of engineering in the project.

SQL

```
CREATE TABLE bpi_daily_result (
  member_id      UUID NOT NULL,
  business_date  DATE NOT NULL,
  matched_pv     NUMERIC(18,4) NOT NULL,
  payout_amount  NUMERIC(18,4) NOT NULL,
  capping_level  INT NOT NULL,
```

```
wash_left      NUMERIC(18,4) NOT NULL DEFAULT 0,
wash_right     NUMERIC(18,4) NOT NULL DEFAULT 0,
carry_left     NUMERIC(18,4) NOT NULL,
carry_right    NUMERIC(18,4) NOT NULL,
created_at     TIMESTAMPTZ NOT NULL DEFAULT NOW(),
PRIMARY KEY (member_id, business_date)
) PARTITION BY RANGE (business_date);
```

5.4 Weekly payout

Friday job aggregates `bpi_daily_result` for the week plus Mentor's Incentive, writes one `payout_batch` + `payout_line` per member, and resets the week counter. Payout lines are immutable once `status = 'PAID'`.

STAGE	STATE	REVERSIBLE?
Aggregate	<code>DRAFT</code>	Yes — recompute freely
Admin approval	<code>APPROVED</code>	Yes — can be voided with an audit reason
Disbursed	<code>PAID</code>	No — only a compensating adjustment entry

06 Mentor's, Startup aur Achievers Engine

6.1 Mentor's Incentive (sponsor tree, 5 levels × 7%)

```
SQL
-- For business_date D: 7% of each downline's BPI payout, up to 5 sponsor levels.
INSERT INTO mentor_incentive (member_id, business_date, level, source_member, amount)
SELECT sc.ancestor_id, r.business_date, sc.level, r.member_id,
       r.payout_amount * 0.07
FROM   bpi_daily_result r
JOIN   sponsor_closure sc ON sc.descendant_id = r.member_id
WHERE  r.business_date = :d AND r.payout_amount > 0
ON CONFLICT (member_id, business_date, source_member) DO NOTHING;
```

CHAIN-BREAK RULE APNE AAP HANDLE HO JAATA HAI

Business rule: agar downline me kisi ko **kisi aur ne** sponsor kiya, wo chain tumhari levels me **nahi** aati. `sponsor_closure` sirf `sponsor_id` links se banti hai, isliye ye rule **by construction** sahi hai — koi extra filter nahi chahiye. Par test zaroor likho: $A \rightarrow B \rightarrow C$ jahan C ko koi teesra sponsor kare, to A ke liye C **level 2 me nahi** aana chahiye.

SET-BASED, ROW-BY-ROW NAHI

Because `sponsor_closure` is capped at depth 5, this is a bounded join — roughly (members with payout) × 5 rows. One statement, no loop, no recursion. This is the payoff for modelling the sponsor tree as its own bounded closure.

Capping-free by design — no cap is applied here, matching the plan. Total ceiling is structural: $5 \times 7\% = 35\%$.

6.2 Startup Incentive (one-time milestones)

```
SQL
-- Fires when cumulative direct PV first crosses a threshold.
CREATE TABLE startup_incentive (
  member_id    UUID NOT NULL REFERENCES members(id),
  milestone_pv  NUMERIC(18,4) NOT NULL,      -- 3000 | 10000
  amount        NUMERIC(18,4) NOT NULL,      -- 2100 | 3000
  awarded_at    TIMESTAMPTZ NOT NULL DEFAULT NOW(),
  PRIMARY KEY (member_id, milestone_pv)      -- one-time, enforced by the PK
);
```

The composite primary key makes 'one-time' a database guarantee. The evaluation runs after the fan-out updates `direct_pv_total`; a simple `INSERT ... ON CONFLICT DO NOTHING` per threshold crossed.

6.3 Achievers (monthly)

STEP	LOGIC
1 · Qualify	Evaluate each member against the rank table using PV — Org-1 PV, Org-2 PV (= left/right subtree volume for the month) and monthly direct-team sales PV.
2 · Pool base	<code>company_turnover = SUM(total_pv) + SUM(total_gv)</code> for the month. Note: PV + GV, not PV alone.
3 · Pool per rank	<code>pool[level] = company_turnover × pct[level]</code> (15/10/8/7/7 %).
4 · Per-achiever	<code>pool[level] ÷ COUNT(qualifiers at level).</code>
5 · Award	Each achiever receives the pools of the last three levels they actually qualified for (L4 → 2+3+4; L5 → 3+4+5; if the top level's criteria were not met, that level is excluded).

VALIDATION KA ANCHOR

With turnover 1,00,00,000 and qualifier counts 50/25/15/10/4, a Level-4 achiever receives 40,000 + 53,333 + 70,000 = **1,63,333 PV** — which matches the figure printed in the client's own plan document. **Use this as a regression test fixture.**

TIMING RULE	IMPLEMENTATION
Closing cycle = calendar month (1st → last day)	Month job <code>business_date</code> range pe chalta hai, <code>NOW()</code> pe nahi — rerun deterministic.
Payout closing ke 10 din ke andar	Payout batch <code>DRAFT</code> me banta hai, admin approve karta hai, phir disburse. SLA alert agar 10 din cross ho.
Tours (hotel/food/conveyance; Goa me conveyance nahi)	Non-cash reward — <code>achiever_award</code> me <code>reward_type</code> aur <code>fulfilment_status</code> track hota hai, ledger me cash entry nahi.

Org-1 / Org-2 monthly volume is derived from `placement_closure` joined to `point_ledger` filtered to the month, grouped by `branch_leg`. For top-of-tree members this is expensive — materialise it into a `member_month_volume` table during the nightly close rather than computing it at month-end.

07 Non-Functional Requirements

7.1 Correctness and auditability

- **Append-only ledgers.** `point_ledger`, `bpi_daily_result`, `payout_line` are never updated. Corrections are compensating rows.
- **Every calculation stores its inputs.** `bpi_daily_result` records the capping level and both carries used, so any past payout can be re-derived and defended.
- **Reconciliation jobs.** Weekly: recompute `member_volume` from the ledger for a random 1% sample plus all disputed members; alert on any drift.
- **Slab versioning.** Capping levels, rank thresholds and percentages are date-effective rows, never constants in code.
- **Immutable business date.** All closes key on `business_date` (IST), not `NOW()`, so a re-run produces identical output.

7.2 Performance target

OPERATION	TARGET	HOW
Bill creation (cashier)	< 300 ms p95	Synchronous work is stock + GST + insert only. Fan-out is async.
Point fan-out per bill	< 1 s p95 (async)	Single closure scan + 2 batched statements.
Genealogy view (1 level)	< 200 ms	Index on (<code>placement_parent_id</code> , <code>leg</code>); lazy-load children.
Downline count / leg volume	< 300 ms	Read from <code>member_volume</code> ; never sum subtrees live.
Nightly close (5 lakh)	< 2 h window	Chunked, parallel, restartable.
Member dashboard	< 500 ms	Redis-cached aggregates, 5-min TTL, invalidated on close.

7.3 Security

- **Money endpoints are idempotent.** Fund request, stock request, payout approval all require an `Idempotency-Key`; the key + response are stored so a retry returns the original result.
- **Server-side authorisation on every request.** Franchise scope and member scope are enforced in the query, never trusted from the client payload.
- **Role-permission matrix in the database** (`roles × permissions`), not hardcoded role checks.

- **Approval flows are two-person** for payouts above a configurable threshold; every approval is written to `audit_log` with actor, timestamp and IP.
- **Secrets server-side only.** Database URL, JWT secret and payment-gateway keys never reach the browser.
- **KYC documents** stored in object storage with signed, short-lived URLs — never public.
- **Rate limiting** on login, OTP and any endpoint that can enumerate member codes.

7.4 Operations

AREA	REQUIREMENT
Backups	Daily full + WAL archiving (PITR). Restore must be tested monthly — an untested backup is not a backup.
Partitions	<code>pg_partman</code> auto-creates next month's partitions. Alert if the future partition is missing. Monthly partitions for 10 years = 120 — well inside the planner's comfortable range.
Pooling	PgBouncer transaction mode , with a separate pool for closing workers so a runaway close cannot starve the storefront. App pool $(\text{cores} \times 2) + 1$ per process.
Monitoring	<code>pg_stat_statements</code> for slow queries; alert on close-job duration, fan-out queue depth, <code>fanout_status='PENDING'</code> age, and replication lag.
Alerts that page	Nightly close failed or exceeded window · payout batch stuck in <code>APPROVED</code> · reconciliation drift detected · fan-out backlog > 10 min.
Environments	dev · staging (with anonymised production-shaped data) · production. The incentive engine must be validated against a full-size synthetic tree before launch.

Teen PostgreSQL trap jo is design me aayenge

1 · A UNIQUE CONSTRAINT ON A PARTITIONED TABLE CANNOT ENFORCE GLOBAL UNIQUENESS

PostgreSQL requires that a unique or primary key on a partitioned table **include all partition key columns**. So `UNIQUE (idempotency_key)` silently becomes `UNIQUE (idempotency_key, business_date)` — which no longer prevents the same key being reused in a different month.

Fix: keep idempotency keys in a small **non-partitioned** table that the transaction inserts into first. The partitioned ledger keeps its composite key for local dedup only.

2 · PGBOUNCER TRANSACTION MODE BREAKS SESSION-LEVEL ADVISORY LOCKS

Transaction pooling breaks `SET/RESET`, `LISTEN`, `PREPARE`, session-scoped temp tables and **session-level advisory locks** — by design. The docs are explicit about this.

Fix: use `pg_advisory_xact_lock` (transaction-scoped, safe) or a `job_lock` row with `SELECT ... FOR UPDATE NOWAIT`. Never `pg_advisory_lock`. Also disable prepared-statement caching in the driver, or run it in a

PgBouncer-compatible mode.

3 · CREATE INDEX CONCURRENTLY IS NOT ALLOWED ON A PARTITIONED TABLE

Build the index `CONCURRENTLY` on each partition individually, then create it on the parent and `ALTER INDEX ... ATTACH PARTITION`. Plan for this before the ledger grows.

7.5 Read replica

- Genealogy browsing, downline reports and rank dashboards read from the replica. The primary serves transactions only.
- `hot_standby = on`; alert when `replay_lag` exceeds ~30 s for 2 minutes.
- Raise `max_standby_streaming_delay` to 60–120 s so long genealogy reports are not cancelled. **Ye behtar hai `hot_standby_feedback enable` karne se** — feedback defers vacuum on the primary and `member_volume` is the hottest-updated table in the system.
- **Never read a payout input from the replica.** Money reads go to the primary, always.

7.6 Storage ka anumaan

OBJECT	ROWS / MONTH	SIZE / MONTH	SIZE / YEAR
<code>bills</code>	75,000	~30 MB	~0.4 GB
<code>point_event</code> (1 per bill)	75,000	~12 MB	~0.15 GB
<code>point_ledger</code> (fan-out, ~42×)	~3.2 M	~350 MB	~4.2 GB
<code>bpi_daily_result</code> + payouts	~3–6 M	0.75–1.5 GB	9–18 GB
<code>placement_closure</code> (static, grows with signups)	20.5 M total	—	~2.4 GB total

CAPACITY KA NATEEJA

Total hot transactional data: **~1.5–2.2 GB/month, 18–27 GB/year**, plus a ~2.4 GB closure table. Detaching partitions older than 24 months keeps the working set under **~50 GB**.

A single well-provisioned primary (**16 vCPU / 64 GB**) handles this with substantial headroom. **The scale problem here is contention and batch-job design, not data volume.**

08 Incentive Engine ki Testing

This is where projects like this fail. The plan is the product — a subtle arithmetic error is a financial liability, not a bug ticket.

8.1 Zaroori test fixtures

FIXTURE	ASSERTS
Client's Day-1 / Day-2 washout example	carry L 1,57,400 → 1,55,400 and carry R 47 → 0. Locks in the old-carry washout rule.
Level-4 achiever = 1,63,333 PV	Locks in the last-3-qualified-levels rule and the PV+GV pool base.
Startup 2,100 + 3,000 = 5,100 (51%)	Locks in the milestone amounts and one-time behaviour.
Mentor 5 × 7% = 35% ceiling	Locks in sponsor-tree depth cap and that no capping is applied.
Extreme-chain placement	From U1 Left → lands at the end of the left chain, never a side branch.
Buyer gets SELF only	No <code>TEAM_*</code> row exists for the buyer from their own bill.
Sponsor gets DIRECT + TEAM	Both rows exist when the sponsor is also a placement ancestor.
Idempotent fan-out	Running the same bill's fan-out twice produces identical ledger state.
Level 0 earns nothing	Self PV 249 with huge team volume → payout 0.

8.2 Load aur soak testing

- Generate a **synthetic tree of 5 lakh members** with realistic skew (long left chains, sparse right legs) before go-live.
- Replay a month of synthetic bills and assert the nightly close completes inside the window.
- Property test the matching function: for random (`carryL`, `carryR`, `todayL`, `todayR`, `cap`), assert PV conservation — `matched + wash + carry_out == avail_in` on both sides.
- Run the close **twice** on the same date and assert the second run is a no-op.

LAUNCH STRATEGY — HAMARI SALAH

Run the point engine in **shadow mode** for 3–4 weeks: points accrue and are visible to members, but **no money is paid**. Members verify their own numbers against their own records.

This surfaces plan misunderstandings while they are still free to fix. Switching on payouts before this validation is the single largest risk in the project.

09 Open Technical Items

#	ITEM	BLOCKS	OWNER
1	Washout trigger — confirm the <code>today_* > 0</code> guard is the intended rule.	BPI engine sign-off	Client
2	Sponsor not in the placement upline — does a team point still accrue, and on which side?	Fan-out logic	Client
3	Washout figure reported: 394 (client sheet) vs 441 (arithmetic).	Report wording only — no state impact	Client
4	Cap-out vs washout — are these two distinct ledgers?	Ledger schema	Client
5	Bill cancellation / stock return — clawback policy for already-paid incentives.	Compensating-entry design	Client
6	Payout disbursement — gateway, frequency, TDS/admin deduction, minimum withdrawal.	Payout module	Client
7	GV and CV semantics — PV rules are precise; GV/CV need the same definition.	Product schema, achiever pool	Client
8	Fund-request rejection — is the deposit returned or does the request simply close?	Wallet state machine	Client
9	Tree depth ceiling — confirm unbounded, or set a practical maximum.	Closure table sizing	Engineering
10	Root-adjacent hot-row contention — measure under load before optimising.	Fan-out throughput	Engineering
11	Price ladder conflict — spec 700/725/750/800 vs handwritten note 670/700/750/800.	Commission engine (data only)	Client
12	Mini-WH buy price ₹685 (margin ₹15) — abhi bhi sahi?	Commission engine	Client
13	<code>retail_profit</code> kabhi withdraw ho payega ya hamesha view-only?	Wallet schema	Client

9.1 Jo decisions lock ho chuke

DECISION	RATIONALE
Closure table over recursive CTE	Ancestor lookup happens ~3.2 M times/month; a single index scan beats a 40-iteration recursion.
Async fan-out via outbox + queue	Keeps cashier latency independent of tree depth; makes retries safe.

Append-only ledgers	Any historical payout must be re-derivable and defensible.
Denormalised leg counters	Subtree summation for top-of-tree members is not viable at this scale.
Monthly RANGE partitioning	Close reads one partition; archival is a detach, not a delete.
BIGINT paise / milli-PV / basis points	NUMERIC exact hai par slow; float to rounding bug ka poora class hai. Split hamesha largest-remainder se.
Slabs as date-effective data	The plan will change; history must remain reproducible.

10 Repo Structure, Apps & Deployment

Client requirement: **ek monorepo** jisme backend, saare frontends aur (future) mobile app ho. Sirf landing page aur user login Google pe index honge — baaki frontends secure/hidden rahenge.

10.1 Monorepo layout

```

TEXT
apolium/
├── apps/
│   ├── web/           # landing + member login/portal    [PUBLIC · SEO indexed]
│   ├── franchise/     # franchise portal                [PRIVATE · noindex]
│   ├── admin/         # super-admin panel              [PRIVATE · noindex + IP lock]
│   ├── api/           # backend (NestJS / FastAPI)
│   └── mobile/        # React Native (future – Phase 7)
├── packages/
│   ├── core/          # ★ BUSINESS RULES – plan engine, pure functions
│   ├── types/         # shared TS types = API contract
│   ├── sdk/           # generated typed API client (web/franchise/admin/mobile use this)
│   ├── ui/            # shared design system components
│   └── config/         # eslint · tsconfig · tailwind preset
├── turbo.json         # task graph + remote cache
├── pnpm-workspace.yaml
└── package.json
  
```

`PACKAGES/CORE` — YE SABSE IMPORTANT DECISION HAI

Poora incentive plan — capping table, BPI matching function, washout rule, mentor %, achiever last-3-levels — **ek hi jagah**, pure functions ke roop me. Koi DB call nahi, koi framework nahi.

Kaun use karega: backend ka nightly close · admin panel ka 'what-if' simulator · member portal ka earnings preview · mobile app · aur unit tests.

Fayda: plan me % ya slab badla → ek file badli → sab jagah lag gaya. Aur agar simulator aur actual payout kabhi alag number dein, wo **bug hai, design nahi** — kyunki dono literally same function call karte hain.

10.2 Apps — kaun kya karta hai

APP	KISKE LIVE	STACK	RENDERING
web	Public + member	Next.js 16 (App Router)	Landing SSG/ISR · portal client-side
franchise	Franchise staff	Next.js 16	Fully client, auth-gated
admin	Super admin	Next.js 16	Fully client, auth-gated

api	Sab	NestJS + Prisma (ya FastAPI)	REST /api/v1
mobile	Member (future)	React Native	Same sdk + core

10.3 Domain aur SEO isolation

APP	DOMAIN	GOOGLE	KAISE ENFORCE HOGA
web	apolium.com	☑ Indexed	sitemap.xml · robots allow · full metadata + JSON-LD
franchise	franchise.apolium.com	✗ Never	X-Robots-Tag: noindex, nofollow header + robots disallow
admin	admin.apolium.com	✗ Never	Same + IP allowlist / VPN + basic-auth at edge
api	api.apolium.com	✗	X-Robots-Tag: noindex · CORS sirf known origins

⚠ SABSE COMMON GALTİ — ROBOTS.TXT INDEXING NAHI ROKTA

robots.txt me Disallow sirf **crawling** rokta hai. Agar Google ko wo URL kahin aur linked mil gaya, wo phir bhi **index kar sakta hai** (bina content ke, sirf URL).

Asli fix: HTTP response header — X-Robots-Tag: noindex, nofollow. Ye har private app pe server-level pe lagega, page-level meta tag pe bharosa mat karo.

Aur noindex ke saath Disallow mat lagao us page pe — agar crawler ko page crawl karne hi nahi diya, to wo noindex header padh hi nahi payega. Private apps pe: **allow crawl, send noindex**.

```

TYPESCRIPT
// apps/franchise/next.config.ts (aur admin me bhi same)
const nextConfig: NextConfig = {
  async headers() {
    return [{
      source: '/(.*?)',
      headers: [
        { key: 'X-Robots-Tag', value: 'noindex, nofollow, noarchive' },
        { key: 'X-Frame-Options', value: 'DENY' },
        { key: 'X-Content-Type-Options', value: 'nosniff' },
        { key: 'Referrer-Policy', value: 'strict-origin-when-cross-origin' },
        { key: 'Strict-Transport-Security',
          value: 'max-age=63072000; includeSubDomains; preload' },
      ],
    }];
  },
};

```

10.4 Auth — teen alag audience, ek hi backend

ASPECT	DESIGN
Token	JWT access (15 min) + refresh (7 din), dono httpOnly cookie me — localStorage me kabhi nahi (XSS).
Cookie scope	Har app ka apna cookie domain/path. Member ka token franchise portal pe kaam nahi karega.
Audience claim	JWT me <code>aud</code> claim — <code>member</code> <code>franchise</code> <code>admin</code> . Backend har route pe verify karta hai. Ek member ka token admin API pe 401 .
Authorization	DB me <code>roles</code> × <code>permissions</code> matrix. Har route handler <code>can(user, module, action)</code> call karta hai. Role check kabhi hardcode nahi.
Scope enforcement	Franchise apna hi data dekhe — ye query me enforce hota hai, client se aayi <code>franchise_id</code> pe kabhi bharosa nahi.
Admin extra	IP allowlist + mandatory 2FA + payout approval pe two-person rule.

10.5 Future-proofing — naya kaam kaise add hoga

KAL YE KARNA PADE...	AAJ KA DESIGN ISSE KAISE AASAN BANATA HAI
Naya incentive type add karna	<code>packages/core</code> me ek naya calculator function likho aur registry me register karo. Close job usko apne aap uthayega. Schema me <code>reason_code</code> already generic hai.
Capping slab / % badalna	<code>capping_levels</code> table me nayi row <code>effective_from</code> ke saath. Koi code deploy nahi . Purane closes apni purani slab se hi reproduce honge.
Mobile app launch	<code>packages/sdk</code> + <code>packages/core</code> already ban chuke hain. App ko sirf UI likhna hai — business logic aur API client reuse.
Naya portal (jaise logistics)	<code>apps/</code> me nayi Next.js app, <code>packages/ui</code> + <code>sdk</code> reuse. Baaki system chhedna nahi padta.
API me breaking change	<code>/api/v1</code> ke saath <code>/api/v2</code> chalega. Purane mobile app users tab tak <code>v1</code> pe chalte rahenge jab tak update na karein.
Naya field har jagah	<code>packages/types</code> me add karo → TypeScript compile error de dega har us jagah jahan handle karna hai. Kuch chhutega nahi.
Tree ki depth badhna	Closure table depth-agnostic hai. Sirf <code>CHECK (depth < 200)</code> guard adjust karna.
Naya level (L4) add	Level ek data row hai, enum nahi. <code>product_level_prices</code> me nayi <code>level_code</code> row.

DO RULES JO ISKO FUTURE-PROOF RAKHTE HAIN

1 • Business rules data me, code me nahi. Slab, %, threshold, level — sab date-effective DB rows.

Code sirf `*engine*` hai, `*plan*` nahi.

2 · Module boundaries = business modules. `franchise/`, `product/`, `billing/`, `network/`, `incentive/` — code ka structure business ke structure jaisa. Naya business module = naya folder, purane ko chhedna nahi padta.

10.6 Deployment

COMPONENT	KAHAN	NOTES
<code>web</code>	Vercel	ISR for landing · edge caching · preview deploy har PR pe
<code>franchise</code> · <code>admin</code>	Vercel (private project)	Password/IP protection · alag env vars
<code>api</code>	Railway / Render / VPS	Docker · health check · graceful SIGTERM drain
Workers	Same host, alag process	Fan-out worker · close job · payout job. Alag PgBouncer pool.
PostgreSQL	Managed (Neon / RDS)	PITR backup · read replica · monthly partitions
Redis	Managed	Queue (BullMQ) + cache + rate limit

CI/CD

Turborepo ka task graph sirf **badle hue apps** build karta hai — `web` me change se `api` rebuild nahi hoga. Remote cache se CI minutes bachte hain.

Pipeline: `lint` → `typecheck` → `test` → `build` → `migrate` → `deploy`. ``packages/core` ke tests fail = pipeline fail`, chahe kisi bhi app me change hua ho. Plan engine ka regression suite yahi rokta hai.

11 Research Basis aur References

The design decisions above are not preferences — each was checked against primary documentation, published production numbers, or vendor benchmarks. This section records what was verified and what remains engineering judgement.

11.1 Tree-storage evaluation

MODEL	INSERT	ALL ANCESTORS	LEG VOLUME	VERDICT
Adjacency list + WITH RECURSIVE	O(1)	O(depth) — ~40 serial lookups	Walks 250k rows — unusable live	Keep as source of truth
Closure table	O(depth), ~41 rows	1 index range scan	Index scan, still 250k rows	Chosen — read model
Materialized path (upline_path bigint[])	O(1) — written once	0 joins	GIN containment	Chosen — co-winner
Nested set (lft/rgt)	O(N) rewrite per signup	O(1)-ish	Fast range query	Disqualified

CLOSURE TABLE AUR MATERIALIZED PATH — DONO KYUN

Because placement is **immutable** — a member is inserted at a slot and essentially never moves — the operation that normally disqualifies materialized path (re-parenting) barely exists here. So we store the upline array on the member row for **zero-join** ancestor reads.

The closure table earns its place separately: it is what makes **set-based batch jobs** possible — 'expand these 1,000 purchases to their ancestors' is a single join the planner can hash and parallelise. A recursive CTE cannot be parallelised and must be re-run per node.

Both together cost ~2.4 GB. Cheap insurance.

Closure table sizing, verified by arithmetic. Rows = $\Sigma(\text{node depth} + 1)$. For 5 lakh members at average depth 40: $500,000 \times 41 \approx 20.5 \text{ M rows}$. With (ancestor_id, descendant_id, depth) as bigints: heap $\approx 1.07 \text{ GB}$ + two B-tree indexes $\approx 0.64 \text{ GB}$ each = **$\approx 2.3\text{--}2.5 \text{ GB total}$** . That fits in the buffer cache of a 32 GB machine. The often-quoted $O(N^2)$ worst case only bites for wide, shallow or arbitrary DAG structures — a binary MLM tree is $O(N \times \text{average depth})$.

11.2 Recursive CTE hot path pe kyun nahi

- PostgreSQL's own docs: "While **RECURSIVE** allows queries to be specified recursively, internally such queries are evaluated **iteratively**." Each level is a separate execution against a working table — **no pipelining, no parallelism across iterations**.

- A recursive CTE is effectively an **optimization fence**: it is materialised, and the planner joins the result to the rest of the query with weak cardinality knowledge. This is why recursive-CTE ancestor lookups produce poor plans once joined to the ledger.
- Cycle risk is real — a corrupted `parent_id` produces non-terminating recursion. Guard with `CHECK (depth < 200)` and PG14+ `CYCLE` detection.
- Honest scale note: for a **single** ancestor fetch both are fine ($\sim 0.2\text{--}1$ ms vs $\sim 50\text{--}150$ μs). The decisive win is set-based batch work, not the single lookup.

11.3 Sources

SOURCE	WHAT IT ESTABLISHED
PostgreSQL 18 docs — Partitioning, WITH Queries, Numeric Types, ltree	Recursive CTEs evaluate iteratively · partitioned UNIQUE must include the partition key · <code>numeric</code> is exact but slow · planner handles a few thousand partitions.
Adyen — “Fighting PostgreSQL write amplification with HOT updates”	Production numbers for <code>fillfactor = 85</code> : HOT rate 63.9% → 92.2%, table 30 → 24 GB, WAL 3.15 → 2.80 TB/day. Also: HOT does not apply to indexed columns.
Modern Treasury — ledger scaling series, “Floats Don’t Work For Storing Cents”	Append-only double-entry, contra entries over edits, entry versioning, and 64-bit integers in minor units for money.
TigerBeetle — “Debit/Credit: The Schema for OLTP”	Double-entry as the minimal complete financial schema; every debit has an equal and opposite credit, enforced at the DB.
Uber — “How LedgerStore Supports Trillions of Indexes”	Immutable ledger at extreme scale; offline validation jobs comparing 760 billion records — the model for our reconciliation discipline.
Stripe — Idempotent Requests API	Store the first response against the key and replay it; compare incoming parameters and reject mismatches. Retention extended here to the payout window, not 24 h.
microservices.io / AWS Prescriptive Guidance — Transactional Outbox	Solves the dual-write problem; guarantees at-least-once , so consumers must be idempotent.
Bill Karwin — “SQL Antipatterns” / closure tables	Adjacency-list-only trees named an antipattern; closure table “generally the simplest for all operations”.
PgBouncer — Features documentation	Transaction pooling breaks <code>SET</code> , <code>PREPARE</code> and session-level advisory locks by design.
TigerData — Postgres INSERT performance benchmarks	<code>INSERT ... UNNEST</code> measured 2.13× faster than <code>VALUES</code> at batch 1,000 (5.02× on a 10-column schema).

Exigo / ByDesign / Epixel (vendor material — directional only)

Volume updates "1 to 5 minutes", commission runs "under 10 minutes", binary pay-leg mechanics. Used as order-of-magnitude targets, not measured facts.

JO ABHI BHI ENGINEERING JUDGEMENT HAI (SOURCE NAHI MILA)

The **~8,000–15,000 aggregated rows per 1,000-bill batch** figure depends on this tree's actual shape — measure it once real data exists.

The **~2 ms per member** close-time estimate is an assumption; benchmark against the synthetic 5-lakh tree.

Preferring a raised `max_standby_streaming_delay` over `hot_standby_feedback` is a judgement call driven by `member_volume` being the hottest-updated table.

Vendor performance claims (Exigo, global-mlm) publish no methodology — treated as directional targets only.

11.4 Ek line me

IS SYSTEM KO ASAL ME KYA TODEGA

At 1–5 lakh members the **data volume is small** — under 50 GB working set. What breaks it is:

- (1)** synchronous 40-row fan-out creating permanent hot-row contention at the top of the tree, and
- (2)** a non-restartable close that double-pays on retry.

Fix those two — with batch-aggregated async fan-out and a chunked idempotent close — and the rest is ordinary PostgreSQL.

*Business rules are specified in the companion **Apolium System Overview (Client)** document. Where the two disagree, the client document is authoritative for **what**, and this document for **how**.*